

FPGA Visual Programming Tool Backend User Manual

Version 0.1.0 | May 2026

Framework: FastAPI + Uvicorn

Language: Python 3.11+

Template Engine: Jinja2

Data Model: Pydantic v2

Table of Contents

1. Project Overview
2. Quick Start
 - 2.1 Installation
 - 2.2 Starting the Server
3. Architecture Overview
 - 3.1 Project Structure
 - 3.2 Data Flow
4. IR Data Model (Intermediate Representation)
5. API Reference
 - 5.1 Health Check
 - 5.2 Code Generation - Top Module
 - 5.3 Code Generation - Testbench
 - 5.4 Parsing - Verilog
 - 5.5 Parsing - VCD Waveform
 - 5.6 Project Management - Save
 - 5.7 Project Management - Load
 - 5.8 Simulation - Compile
 - 5.9 Simulation - Run
6. Running Tests
7. Demo: Quick Workflow
8. Appendix: Dependencies

1. Project Overview

The FPGA Visual Programming Tool backend provides a REST API service for the visual FPGA design workflow. It handles Verilog/VHDL code generation, hardware description parsing, simulation orchestration, and waveform analysis.

Core Capabilities

- IR-based Verilog Top module generation (auto port inference, wire declaration, instantiation)
- Testbench scaffolding with clock generation, reset sequence, stimulus placeholders
- Verilog file parsing for importing existing projects (regex-based, demo quality)
- VCD waveform file parsing for simulation result visualization
- ModelSim/Quarta simulation integration (vlib/vlog/vsim via subprocess)
- Project file save/load (.fpga.json format)

2. Quick Start

2.1 Installation

Requirements: Python 3.11 or newer.

```
cd backend
pip install -r requirements.txt
```

Dependencies include: FastAPI, Uvicorn, Pydantic v2, Jinja2, pytest, httpx.

2.2 Starting the Server

Development mode (with hot reload):

```
cd backend
python main.py
# or:
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

The server starts at <http://localhost:8000>. Open <http://localhost:8000/docs> for the auto-generated Swagger API documentation, or <http://localhost:8000/redoc> for ReDoc.

3. Architecture Overview

3.1 Project Structure

```

backend/
main.py           # FastAPI app entry point
requirements.txt  # Python dependencies
pyproject.toml    # Pytest configuration
demo_input.json  # Sample IR input
app/
  routes.py       # All API endpoint definitions
  models/
    ir.py         # IR data models (Pydantic)
  generators/
    top.py        # Top-module Verilog generator
    testbench.py  # Testbench generator
  templates/
    top.v.j2      # Top module Jinja2 template
    testbench.v.j2 # Testbench Jinja2 template
  parser/
    verilog_parser.py # Verilog parsing (regex)
  simulator/
    base.py       # Abstract simulator interface
    modelsim.py   # ModelSim simulator backend
  utils/
    vcd_parser.py # VCD waveform parser
  tests/
    test_ir.py    # IR model tests
    test_generators.py # Generator tests
    test_vcd.py   # VCD parser tests
    test_api.py   # API endpoint tests

```

3.2 Data Flow

The frontend canvas produces an IR (Intermediate Representation) JSON object describing modules, ports, and connections. This IR is the single exchange format between the frontend and backend. The backend processes the IR and generates Verilog code, parses input files, or orchestrates simulation.

```

Frontend Canvas
|
v
IR (JSON) --POST--> /api/generate/top
|
Jinja2 Template <---+--- Port inference
|      Wire declaration
|      Instantiation
v
Verilog (.v)

```

4. IR Data Model (Intermediate Representation)

All Pydantic v2 models are defined in `app/models/ir.py`. The IR is versioned for forward/backward compatibility.

Core Types

Port -- defines a module port (name, direction, width, signed, array_size)

Module -- a design node on the canvas (id, name, type, ports[], position, config{})

Connection -- an edge between two ports (src_module, src_port, dst_module, dst_port, wire_name)

WrappedModule -- a user-encapsulated sub-design (name, exposed_ports[], internal_modules[], internal_connections[])

IR -- the top-level intermediate representation (version, modules[], connections[], wrapped_modules[], top_module_name)

ProjectFile -- complete project save file (version, name, ir, board, simulation, canvas_state)

SimulationConfig -- simulation parameters (simulator, clock_period_ns, reset_cycles, sim_time_us, monitored_signals[])

BoardConfig -- board-specific IP configuration (board_name, fpga_part, clock_pins{}, gpio_map{}, constraints)

Module Types

- base: standard user-defined or imported module
- wrapped: encapsulated sub-circuit (can expand/collapse)
- board_ip: vendor-provided IP core (PLL, GPIO, UART, etc.)

5. API Reference

Base URL: <http://localhost:8000/api>. All requests use JSON. Responses return {"success": bool, ...}.

5.1 GET /api/health

Health check endpoint. Returns service status.

```
GET /api/health
Response: {"status": "ok", "service": "fpga-visual-tool-backend"}
```

5.2 POST /api/generate/top

Generate Verilog top-module code from an IR object.

```
POST /api/generate/top
Body: { "ir": { IR object } }
Response: { "success": true, "verilog": "..." }
```

The generator automatically: (a) infers top-level ports from unconnected module ports; (b) creates wire declarations for all connections; (c) instantiates all sub-modules with proper port mapping; and (d) leaves unused ports disconnected with comments.

5.3 POST /api/generate/testbench

Generate a simulation testbench framework from an IR and simulation config.

```
POST /api/generate/testbench
Body: { "ir": { IR }, "simulation": { SimulationConfig } }
Response: { "success": true, "verilog": "..." }
```

The testbench includes: clock generation, configurable reset sequence, DUT instantiation, stimulus placeholder section, \$dumpfile/\$dumpvars for waveform export, and \$finish at end of simulation.

5.4 POST /api/parse/verilog

Parse a Verilog file or directory, extracting module definitions and instantiation relationships.

```
POST /api/parse/verilog
Body: { "file_path": "/path/to/file.v" }
Response: { "success": true, "modules": [...] }
```

Supports .v and .sv extensions. This is a regex-based parser (demo quality); for production use, integrate pyverilog or sv-parser for full SystemVerilog AST support.

5.5 POST /api/parse/vcd

Parse a VCD waveform file into structured JSON for frontend visualization.

```
POST /api/parse/vcd
Body: { "file_path": "/path/to/waveform.vcd" }
Response: { "success": true, "waveform": { "signals": [...], ... } }
```

Each signal in the response contains a changes[] array with {time_ns, value} pairs, plus metadata (width, signed). Supports \$var, \$dumpvars, #time markers, multi-bit bus values.

5.6 POST /api/project/save

Save a complete project to a .fpga.json file on disk.

```
POST /api/project/save
Body: { "project": { ProjectFile }, "file_path": "..." }
Response: { "success": true, "path": "..." }
```

5.7 POST /api/project/load

Load a .fpga.json project file from disk.

```
POST /api/project/load
Body: { "file_path": "/path/to/project.fpga.json" }
Response: { "success": true, "project": { ProjectFile } }
```

5.8 POST /api/simulate/compile

Compile HDL source files using ModelSim/Questa (requires local installation).

```
POST /api/simulate/compile
Body: { "sources": ["src1.v", "src2.v"], "work_dir": "./sim_work" }
Response: { "success": true, "stdout": "...", "errors": [], "warnings": [] }
```

Internally executes vlib work + vlog -work work <source> for each source file. Compilation errors and warnings are captured and returned.

5.9 POST /api/simulate/run

Run simulation using ModelSim/Questa (requires local installation).

```
POST /api/simulate/run
Body: { "top_module": "my_top", "sim_time": "1us",
        "work_dir": "./sim_work" }
Response: { "success": true, "vcd_path": "...", "stdout": "..." }
```

Optionally accepts a sources[] list to compile first, then run. Internally generates a TCL script with vcd add commands and executes vsim -c -do. The simulation output VCD file path is returned.

6. Running Tests

The project includes 23 tests across 4 test files. Test framework: pytest + pytest-asyncio + httpx.

Run All Tests

```
cd backend
python -m pytest tests/ -v
```

Run Specific Test File

```
python -m pytest tests/test_ir.py -v
python -m pytest tests/test_generators.py -v
python -m pytest tests/test_vcd.py -v
python -m pytest tests/test_api.py -v
```

Test Summary

test_ir.py (9 tests): Validates Port, Module, Connection, IR, ProjectFile model creation, serialization, and deserialization.

test_generators.py (5 tests): Validates TopGenerator output (port inference, wire generation, instantiation) and TestbenchGenerator output (clock config, DUT, dump statements).

test_vcd.py (4 tests): Validates VCD parser against sample VCD content (single-bit signals, multi-bit buses, empty files).

test_api.py (5 tests): End-to-end HTTP tests against all API endpoints using httpx AsyncClient with ASGI transport (no server needed).

7. Demo: Quick Workflow

A complete end-to-end demo using the sample file demo_input.json.

Step 1 - Start the server

```
cd backend
python main.py
```

Step 2 - Generate Top module

```
curl -X POST http://localhost:8000/api/generate/top \
-H "Content-Type: application/json" \
-d @demo_input.json
```

This sends an IR with two modules (clk_divider, led_controller) and one connection. The backend returns a complete Verilog top module with inferred ports (clk_in, rst_n as inputs; led_out as output), a wire declaration for clk_divided, and two module instantiations.

Step 3 - Generate Testbench

```
curl -X POST http://localhost:8000/api/generate/testbench \
-H "Content-Type: application/json" \
-d '{"ir":{"version":"0.1.0","modules":[],"connections":[],"wrapped_modules":[],"top_module_name":'`
```

Step 4 - Browse API docs

FPGA Visual Programming Tool - Backend Manual

Open <http://localhost:8000/docs> in a browser for interactive Swagger UI with all endpoints.

8. Appendix: Dependencies

fastapi	>= 0.111.0	Web framework
uvicorn	>= 0.29.0	ASGI server
pydantic	>= 2.7.0	Data validation
jinja2	>= 3.1.4	Template engine
vcdvcd	>= 2.0.0	VCD library (auxiliary)
pytest	>= 8.2.0	Test framework
pytest-asyncio	>= 0.23.7	Async test support
httpx	>= 0.27.0	HTTP test client

For simulation features, ModelSim/Quarta must be installed separately and available on the system PATH. The simulator module uses subprocess calls to vlib, vlog, vcom, and vsim executables.

The VCD parser (app/utis/vcd_parser.py) is a self-contained implementation that does not depend on the vcdvcd library. It handles \$timescale, \$var, \$dumpvars, #time markers, and single/multi-bit value changes.